

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería
Análisis y Diseño de Algoritmos
Semestre I – 2026



Propuesta técnica para la gestión de recursos en un data center

Integrantes:

Nájera Marakovits, Ingrid Nina Alessandra – 231088

Ramírez, Diego – 23601

1. Crisis 1: Programación de tareas

Se tiene un conjunto de tareas de duración 1. Cada tarea tiene un deadline d_i y una penalización w_i si no se completa a tiempo. El objetivo es organizar las tareas para que la penalización total de las que quedan fuera sea la menor posible.

Propuesta

La idea es darle prioridad a las tareas cuya pérdida cuesta más. Como todas duran lo mismo, lo más importante no es cuánto tardan, sino cuánto se pierde si no se hacen.

Si T es el conjunto de tareas que sí se logran ejecutar a tiempo, entonces la penalización total es

$$\sum_{a_i \notin T} w_i = \sum_{i=1}^n w_i - \sum_{a_i \in T} w_i.$$

Como la suma total de penalizaciones es fija, minimizar lo que se pierde equivale a maximizar la suma de las penalizaciones de las tareas que sí se salvan.

Por eso, el criterio elegido es ordenar las tareas de mayor a menor penalización. La función de orden es

$$f(a_i) = w_i.$$

Luego se recorre la lista en ese orden y cada tarea se coloca en el último espacio libre que no pase de su deadline. Si no hay espacio, se descarta.

Criterios que parecen buenos pero fallan

Un criterio posible sería ordenar por deadline más pequeño primero. El problema es que eso puede dar prioridad a tareas urgentes pero poco importantes.

Cuadro 1: Ejemplo donde ordenar por deadline no da la mejor solución

Tarea	Deadline	Penalización
a_1	1	2
a_2	2	100
a_3	2	99

Si se sigue ese criterio, es fácil terminar haciendo a_1 y una de las otras dos, dejando fuera una tarea muy costosa. La mejor decisión realmente es salvar a_2 y a_3 , porque así la penalización que queda es mucho menor.

Otro criterio que también falla es dar prioridad a las tareas con más holgura o con deadline más grande.

Cuadro 2: Ejemplo donde dar prioridad a tareas menos urgentes falla

Tarea	Deadline	Penalización
a_1	1	100
a_2	2	2
a_3	2	1

Si primero se hacen las tareas con deadline 2, la tarea a_1 queda fuera, aunque era la más costosa de perder. Entonces ese criterio tampoco protege lo que realmente importa.

Justificación matemática

Para justificar formalmente la elección, se modela el problema como un matroide $M = (S, \mathcal{I})$, donde $\mathcal{I}$ es la familia de subconjuntos de tareas que sí pueden ordenarse de forma que todas terminen a tiempo.

Una forma útil de describir la factibilidad es esta: un conjunto X de tareas es factible si, para todo tiempo t , la cantidad de tareas de X cuyo deadline es menor o igual que t no supera a t . En símbolos,

$$|\{a_i \in X : d_i \leq t\}| \leq t \quad \text{para todo } t.$$

Esto significa que antes del tiempo t no se pueden haber acumulado más de t tareas que exijan terminar a más tardar en ese momento.

Primero, si un conjunto de tareas es factible, entonces cualquier subconjunto suyo también lo es. En efecto, si $A \in \mathcal{I}$ y $B \subseteq A$, entonces para cualquier tiempo t se cumple

$$|\{a_i \in B : d_i \leq t\}| \leq |\{a_i \in A : d_i \leq t\}| \leq t.$$

Por lo tanto, B también es factible. Esa es la propiedad hereditaria.

Ahora hay que ver la propiedad de intercambio. Sean A y B dos conjuntos factibles con $|A| < |B|$. Queremos ver que existe alguna tarea de $B \setminus A$ que puede agregarse a A sin perder factibilidad.

Supongamos que eso no es cierto. Entonces cada tarea $x \in B \setminus A$ haría que $A \cup \{x\}$ dejara de ser factible. Eso significa que para cada una de esas tareas existe algún tiempo t_x tal que al agregarla se rompe la condición

$$|\{a_i \in A \cup \{x\} : d_i \leq t_x\}| \leq t_x.$$

Como A ya era factible, la única forma de que falle al agregar x es que antes ya hubiera exactamente t_x tareas de A con deadline menor o igual que t_x , y además que $d_x \leq t_x$. En otras palabras,

$$|\{a_i \in A : d_i \leq t_x\}| = t_x.$$

Tomemos ahora un tiempo cualquiera t . Si hubiera una tarea $x \in B \setminus A$ con $d_x \leq t$, entonces por el argumento anterior el conjunto A ya tendría t tareas con deadline menor o igual que t . Eso implica que

$$|\{a_i \in B : d_i \leq t\}| > |\{a_i \in A : d_i \leq t\}| = t,$$

lo cual contradice que B sea factible.

Así, la suposición inicial no puede ser cierta. Entonces sí existe al menos una tarea $x \in B \setminus A$ tal que $A \cup \{x\}$ sigue siendo factible. Con eso se cumple la propiedad de intercambio.

Como se cumplen la propiedad hereditaria y la propiedad de intercambio, el modelo sí tiene la estructura necesaria.

Después, como las tareas se recorren en orden de mayor a menor penalización y solo se agregan si siguen siendo factibles, se obtiene un conjunto factible con peso máximo. La idea es que si una solución óptima no contiene alguna tarea que el algoritmo escogió primero, esa tarea puede intercambiarse por otra de menor o igual penalización sin perder factibilidad. Repitiendo ese intercambio, la solución óptima puede transformarse hasta coincidir con la construida por el algoritmo, sin empeorar su valor. En otras palabras, se salvan primero las tareas cuya pérdida sería más cara, y eso produce una solución óptima.

Complejidad

Primero se ordenan las tareas, lo cual cuesta $O(n \log n)$. Después se asignan a espacios válidos. Con una implementación eficiente, el tiempo total sigue siendo

$$O(n \log n).$$

Por qué otros enfoques no convienen

Probar todos los subconjuntos posibles tendría costo exponencial y no sería práctico. También se podría pensar en programación dinámica, pero aquí no hace falta, porque la estructura del problema ya permite resolverlo de forma directa con una estrategia mucho más simple y eficiente.

2. Crisis 2: Asignación de memoria a máquinas virtuales

Se tiene una capacidad máxima de memoria W . Cada máquina virtual tiene un consumo de memoria w_i y un valor v_i . Cada una se toma completa o no se toma.

Propuesta

En este caso la mejor opción es programación dinámica. Aquí no conviene tomar decisiones solo por intuición local, porque elegir una máquina puede bloquear combinaciones mejores más

adelante.

Por qué una estrategia greedy no es suficiente

Una idea tentadora sería escoger primero la máquina con mejor relación valor/memoria. El problema es que eso no siempre lleva a la mejor combinación final.

Cuadro 3: Ejemplo donde una decisión local no da el mejor resultado

VM	Memoria	Valor	v_i/w_i
VM1	10	60	6
VM2	20	100	5
VM3	30	120	4

Si la capacidad es 50, ese criterio tomaría primero VM1 y luego VM2, con valor total 160. Pero la mejor solución es VM2 junto con VM3, que da valor total 220.

Ese ejemplo muestra que una decisión local puede fallar, pero no es la única razón. El problema de fondo es estructural. Como cada máquina virtual es indivisible, no se puede tomar solo una parte para ajustar mejor la capacidad. Eso hace que una decisión temprana pueda bloquear combinaciones futuras mejores.

También se puede ver que aquí no aparece la misma estructura de la Crisis 1. Si se toma capacidad $W = 4$ y máquinas con memorias 3, 2 y 2, entonces los conjuntos

$$A = \{\text{VM de memoria 3}\} \quad \text{y} \quad B = \{\text{VM de memoria 2, VM de memoria 2}\}$$

son factibles, porque en ambos casos la memoria total no supera 4. Sin embargo, $|A| < |B|$ y no se puede agregar a A ninguna máquina de $B \setminus A$, porque $3 + 2 > 4$. Entonces aquí no se cumple la propiedad de intercambio.

Eso explica por qué una estrategia greedy no tiene una base confiable en este problema. En la versión fraccionaria de la mochila sí funciona escoger por relación valor/peso, porque se pueden tomar partes de un objeto. Aquí eso no se puede hacer, y por eso una buena decisión local no garantiza una buena solución global.

Subestructura óptima

Sea $OPT(i, c)$ el mejor valor que se puede obtener usando solo las primeras i máquinas y teniendo capacidad disponible c .

Para decidir qué pasa con la máquina i , hay dos casos.

Si no se toma, entonces el mejor valor es exactamente el mismo que usando las primeras $i - 1$ máquinas con capacidad c :

$$OPT(i, c) = OPT(i - 1, c).$$

Si sí se toma, entonces debe cumplirse que $w_i \leq c$. En ese caso se gana el valor v_i y el resto del problema queda reducido a usar las primeras $i - 1$ máquinas con capacidad $c - w_i$:

$$OPT(i, c) = v_i + OPT(i - 1, c - w_i).$$

La razón por la que esto funciona es que, si una solución óptima para el problema grande incluye a la máquina i , entonces lo que queda después de tomarla también tiene que ser óptimo para el subproblema restante. Si no lo fuera, se podría reemplazar por una solución mejor y entonces la solución original tampoco sería óptima.

Entonces la relación final es

$$OPT(i, c) = \begin{cases} 0, & \text{si } i = 0 \text{ o } c = 0, \\ OPT(i - 1, c), & \text{si } w_i > c, \\ \text{máx}\{OPT(i - 1, c), v_i + OPT(i - 1, c - w_i)\}, & \text{si } w_i \leq c. \end{cases}$$

Esta relación funciona porque una solución óptima para el problema grande se construye a partir de soluciones óptimas de problemas más pequeños.

Algoritmo y complejidad

Se construye una tabla de tamaño $(n + 1) \times (W + 1)$, donde cada celda guarda el mejor valor posible para una cantidad de máquinas y una capacidad dada.

Como cada celda se llena una sola vez y su cálculo toma tiempo constante, el tiempo total es

$$O(nW).$$

El espacio también es

$$O(nW).$$

En cambio, una solución recursiva simple, sin guardar resultados intermedios, repite muchos cálculos y en el peor caso toma tiempo exponencial:

$$O(2^n).$$

Por qué otros enfoques no convienen

Greedy no da garantía de optimalidad, porque una buena decisión local no siempre lleva a la mejor combinación total. Fuerza bruta sí encontraría la respuesta correcta, pero con un costo demasiado alto. Programación dinámica sí aprovecha la estructura del problema y permite encontrar la mejor solución de manera exacta.

Implementación

La implementación funcional de esta parte se entrega en el repositorio indicado al inicio.

```
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VII SEMESTRE\Análisis y Diseño de Algoritmos\ADA> & c:\Program Files\Python37\python.exe "C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VII SEMESTRE\Análisis y Diseño de Algoritmos\ADA\crisis2.py"
```

```
Capacidad máxima de memoria: 50
```

```
VMs disponibles:
```

- VM1: memoria = 10, valor = 60
- VM2: memoria = 20, valor = 100
- VM3: memoria = 30, valor = 120
- VM4: memoria = 25, valor = 75

```
Valor óptimo total: 220
```

```
Memoria total usada: 50
```

```
VMs seleccionadas:
```

- VM2: memoria = 20, valor = 100
- VM3: memoria = 30, valor = 120

```
PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VII SEMESTRE\Análisis y Diseño de Algoritmos\ADA>
```

Figura 1: Salida del programa para la Crisis 2, mostrando la capacidad disponible, el valor óptimo y las máquinas virtuales seleccionadas.

3. Síntesis

La diferencia principal entre ambas crisis no está solo en el tema, sino en la estructura del problema.

En la Crisis 1 sí existe una base que permite tomar decisiones locales sin arruinar la solución completa. Esa base se formaliza con matroides. Gracias a esa estructura, elegir primero las tareas con mayor penalización sí puede justificarse matemáticamente.

En la Crisis 2 eso no pasa. Aunque el problema sí puede dividirse en partes más pequeñas bien definidas, una decisión local no garantiza que después todavía quede espacio para la mejor combinación. Por eso aquí se necesita programación dinámica.

También queda claro que en un algoritmo greedy no basta con decir voy a ordenar y escoger. El criterio de orden importa mucho. En la Crisis 1 había criterios que parecían razonables, como usar solo el deadline, pero daban malas respuestas. Entonces no solo importa que el problema tenga la estructura correcta, sino también elegir el criterio correcto dentro de esa estructura.